Guía 1 de Ejercicios

K-Nearest Neighbors (KNN)

Universidad de SanAndrés

Ejercicios

1 · Verdadero o Falso

1. KNN es un algoritmo de aprendizaje supervisado que requiere entrenamiento previo.
2. El parámetro k siempre debe ser impar para evitar empates en clasificación.
3. KNN es sensible a la escala de las variables de entrada.
4. Un valor de k muy grande puede llevar a overfitting.
5. La distancia euclidiana es la única métrica que se puede usar en KNN.
6. KNN puede manejar tanto clasificación como regresión.
7. El algoritmo KNN almacena todos los datos de entrenamiento en memoria.
8. Un valor de $k = 1$ siempre clasifica correctamente todos los puntos de entrenamiento.
9. KNN es un algoritmo paramétrico.
10. La normalización de variables es opcional en KNN.

2 · Opción múltiple

1. ¿Cuál de las siguientes NO es una ventaja de KNN?
   a. No requiere entrenamiento
   b. Simple de entender
   c. Maneja datos faltantes automáticamente
   d. No hace suposiciones sobre la distribución de datos
2. Para un problema de clasificación binaria con k=3, si los 3 vecinos más cercanos son [1, 0, 1], la predicción será:
   a. 0
   b. 1
   c. No se puede determinar
   d. Error del algoritmo
3. ¿Qué sucede si k es igual al número total de puntos de entrenamiento?
   a. El algoritmo falla
   b. Siempre predice la clase mayoritaria
   c. No es posible
   d. Predice la clase del punto más cercano
4. ¿Cuál es el efecto de aumentar k en un problema de clasificación?
   a. Mayor overfitting
   b. Mayor underfitting

c. No tiene efecto

d. Depende del dataset

5. ¿Qué métrica de distancia es más apropiada para variables categóricas?

   a. Euclidiana

   b. Manhattan

   c. Hamming

   d. Minkowski

3 · Clasificación simple

Tenemos los siguientes datos de entrenamiento para clasificar flores:

Longitud pétalo	Ancho pétalo	Clase
1.4	0.2	Setosa
1.4	0.2	Setosa
1.3	0.2	Setosa
4.7	1.4	Versicolor
4.5	1.5	Versicolor
4.9	1.5	Versicolor
6.0	2.5	Virginica
5.1	1.9	Virginica
5.9	2.1	Virginica

Un nuevo punto tiene longitud de pétalo = 4.8 y ancho de pétalo = 1.6.

1. Calculá la distancia euclidiana a todos los puntos de entrenamiento.
2. Encontrá los 3 vecinos más cercanos (k=3).
3. ¿Cuál es la clasificación del nuevo punto?

4 · Efecto del parámetro k

Usando los mismos datos del ejercicio anterior:

1. Clasificá el punto (4.8, 1.6) usando k=1.
2. Clasificá el mismo punto usando k=5.
3. Clasificá el mismo punto usando k=7.
4. ¿Por qué pueden diferir los resultados?

5 · Normalización y múltiples variables

Tenemos datos de clientes bancarios:

Edad	Ingresos	Deuda	Clase
25	30000	5000	Riesgo Bajo
30	45000	8000	Riesgo Bajo

35	40000	12000	Riesgo Bajo
45	35000	25000	Riesgo Alto
50	75000	30000	Riesgo Alto
55	55000	35000	Riesgo Alto

1. Calculá las distancias al punto (38, 70000, 15000) sin normalizar.
2. Encontrá los 3 vecinos más cercanos sin normalizar y clasificá el nuevo cliente.
3. Normalizá los datos.
4. Encontrá los 3 vecinos más cercanos al punto normalizado y clasificá el nuevo cliente.
5. ¿Cómo cambian los resultados con y sin normalización?

6 · Datos desbalanceados

Tenemos un dataset con 100 puntos de clase A y 10 puntos de clase B:

1. Si k=5, ¿qué problema puede surgir?
2. ¿Cómo se puede solucionar este problema?
3. ¿Qué valor de k sería más apropiado?

7 · Curse of dimensionality

1. Explicá qué es la curse of dimensionality.
2. ¿Por qué afecta especialmente a KNN?
3. ¿Qué técnicas se pueden usar para mitigarla?

8 · Métricas de distancia

Tenemos dos puntos: A(1, 2, 3) y B(4, 6, 9).

1. Calculá la distancia euclidiana.
2. Calculá la distancia Manhattan.
3. Calculá la distancia de Minkowski con p=3.
4. ¿Cuál es la relación entre estas distancias?

9 · Pesos por distancia

En lugar de votación simple, usamos pesos inversos a la distancia:

1. Si las distancias son [2, 4, 6] y las clases [A, B, A], calculá la predicción con pesos.
2. Compará con la votación simple.
3. ¿Cuándo es útil usar pesos?

10 · Regresión simple

Tenemos datos de precios de casas:

Área (m²)	Precio (USD)
50	80000
60	95000

70	110000
80	125000
90	140000
100	155000

1. Predecí el precio de una casa de 75 m² usando k=3.
2. ¿Cuál es la diferencia con clasificación?

11 · Regresión con múltiples variables

Tenemos datos de ventas:

Publicidad	Precio	Ventas
1000	10	500
1500	12	600
2000	15	700
2500	18	800
3000	20	900

1. Normalizá los datos usando min-max scaling.
2. Calculá las distancias del punto (1800, 14) a todos los puntos normalizados.
3. Encontrá los 3 vecinos más cercanos.
4. Predecí las ventas para publicidad=1800, precio=14 usando k=3.
5. ¿Cómo cambiaría la predicción si usaras k=5?

12 · Outliers en regresión

Tenemos datos de precios de casas con un outlier:

Área (m²)	Precio (USD)	
50	80000	
60	95000	
70	110000	
80	125000	
90	140000	
100	155000	
95	500000	outlier

1. Predecí el precio de una casa de 85 m² usando k=3 con y sin el outlier.
2. ¿Cómo afectan los outliers a KNN en regresión?
3. ¿Qué técnicas se pueden usar para mitigar su efecto?

13 · El rol de k: sesgo vs. varianza

Este es el hiperparámetro central de KNN. Pensá qué pasa en los dos extremos:

1. Con $k = 1$, ¿cómo queda la frontera de decisión? ¿Qué error de entrenamiento tiene? ¿Es sesgo alto o varianza alta?
2. Con $k = n$ (todos los puntos), ¿qué predice el modelo para cualquier entrada? ¿Sesgo alto o varianza alta?
3. ¿Cómo se elige un k intermedio en la práctica? ¿Qué relación tiene esto con overfitting y underfitting?

14 · Distancia para variables categóricas

Cuando las variables son categóricas, la distancia euclidiana no aplica: se usa la **distancia de Hamming**, que cuenta en cuántos atributos difieren dos observaciones.

Obs	Color	Tamaño	Categoría
1	Rojo	Chico	Fruta
2	Rojo	Grande	Fruta
3	Azul	Chico	Juguete
4	Verde	Grande	Juguete

1. Calculá la distancia de Hamming (sobre Color y Tamaño) de la nueva observación X = (Rojo, Chico) a cada una de las cuatro.
2. Clasificá X con $k = 1$ y con $k = 3$.
3. ¿Por qué no tendría sentido usar la distancia euclidiana acá?

15 · Lazy learning y costo computacional

Se dice que KNN es un algoritmo "lazy" (perezoso): no construye un modelo en el entrenamiento.

1. ¿Qué hace KNN exactamente durante el "entrenamiento"?
2. Si el set de entrenamiento tiene n observaciones, ¿cuál es el costo de clasificar **una** nueva observación? ¿Por qué?
3. Comparado con un árbol de decisión ya entrenado, ¿cuál es más caro en predicción y cuál en entrenamiento?
4. ¿Qué estructuras o técnicas se pueden usar para acelerar la búsqueda de vecinos?

Soluciones

1 · Verdadero o Falso

1. **Falso.** KNN no requiere entrenamiento previo, es un algoritmo lazy.
2. **Falso.** k puede ser par, pero impar evita empates en clasificación binaria.
3. **Verdadero.** KNN es muy sensible a la escala de las variables.
4. **Falso.** Un k muy grande lleva a underfitting, no overfitting.
5. **Falso.** Se pueden usar múltiples métricas de distancia.
6. **Verdadero.** KNN funciona para ambos tipos de problemas.
7. **Verdadero.** KNN almacena todos los datos de entrenamiento.
8. **Verdadero.** Con k=1, cada punto se clasifica como su vecino más cercano (él mismo).
9. **Falso.** KNN es no paramétrico.
10. **Falso.** La normalización es crucial en KNN.

2 · Opción múltiple

1. **c)** KNN no maneja datos faltantes automáticamente.
2. **b)** 1 (mayoría de votos).
3. **b)** Siempre predice la clase mayoritaria.
4. **d)** Depende del dataset específico.
5. **c)** Hamming es apropiada para variables categóricas.

3 · Clasificación simple

1) Distancias euclidianas al punto (4.8, 1.6):

- (1.4, 0.2) Setosa: $\sqrt{(4.8-1.4)^2 + (1.6-0.2)^2} = \sqrt{11.56 + 1.96} = \mathbf{3.68}$
- (1.4, 0.2) Setosa: $\sqrt{(4.8-1.4)^2 + (1.6-0.2)^2} = \sqrt{11.56 + 1.96} = \mathbf{3.68}$
- (1.3, 0.2) Setosa: $\sqrt{(4.8-1.3)^2 + (1.6-0.2)^2} = \sqrt{12.25 + 1.96} = \mathbf{3.77}$
- (4.7, 1.4) Versicolor: $\sqrt{(4.8-4.7)^2 + (1.6-1.4)^2} = \sqrt{0.01 + 0.04} = \mathbf{0.22}$
- (4.5, 1.5) Versicolor: $\sqrt{(4.8-4.5)^2 + (1.6-1.5)^2} = \sqrt{0.09 + 0.01} = \mathbf{0.32}$
- (4.9, 1.5) Versicolor: $\sqrt{(4.8-4.9)^2 + (1.6-1.5)^2} = \sqrt{0.01 + 0.01} = \mathbf{0.14}$
- (6.0, 2.5) Virginica: $\sqrt{(4.8-6.0)^2 + (1.6-2.5)^2} = \sqrt{1.44 + 0.81} = \mathbf{1.50}$
- (5.1, 1.9) Virginica: $\sqrt{(4.8-5.1)^2 + (1.6-1.9)^2} = \sqrt{0.09 + 0.09} = \mathbf{0.42}$
- (5.9, 2.1) Virginica: $\sqrt{(4.8-5.9)^2 + (1.6-2.1)^2} = \sqrt{1.21 + 0.25} = \mathbf{1.21}$

2) Los 3 vecinos más cercanos: (4.9, 1.5) Versicolor (0.14), (4.7, 1.4) Versicolor (0.22), (4.5, 1.5) Versicolor (0.32).

3) Clasificación: **Versicolor** (3 votos).

4 · Efecto del parámetro k

1) Con k=1: **Versicolor** (punto más cercano, 0.14).

2) Con k=5: los 5 vecinos son 3 Versicolor y 2 Virginica (5.1,1.9 y 5.9,2.1). Resultado: **Versicolor** (3 vs 2).

3) Con k=7: 3 Versicolor, 3 Virginica, 1 Setosa. Resultado: **Versicolor** (desempate por el más cercano).

4) Con k=1 solo se considera el vecino más cercano; con k más grande entran puntos de otras clases que pueden cambiar la mayoría. Acá Versicolor gana en los tres casos porque el nuevo punto cae claramente en su región.

5 · Normalización y múltiples variables

1) Distancias sin normalizar al punto P(38, 70000, 15000). Domina la variable ingresos por su escala:

- A(25, 30000, 5000): $\sqrt{13^2 + 40000^2 + 10000^2} = \mathbf{41231}$
- B(30, 45000, 8000): $\sqrt{8^2 + 25000^2 + 7000^2} = \mathbf{25962}$
- C(35, 40000, 12000): $\sqrt{3^2 + 30000^2 + 3000^2} = \mathbf{30150}$
- D(45, 35000, 25000): $\sqrt{7^2 + 35000^2 + 10000^2} = \mathbf{36401}$
- E(50, 75000, 30000): $\sqrt{12^2 + 5000^2 + 15000^2} = \mathbf{15811}$
- F(55, 55000, 35000): $\sqrt{17^2 + 15000^2 + 20000^2} = \mathbf{25000}$

2) Vecinos: E (Riesgo Alto), F (Riesgo Alto), B (Riesgo Bajo). Clasificación: **Riesgo Alto** (2 vs 1).

3) Normalizando (P → 0.43, 0.89, 0.33):

Punto	Edad	Ingresos	Deuda	Clase
A	0.00	0.00	0.00	RB
B	0.17	0.33	0.10	RB
C	0.33	0.22	0.23	RB
D	0.67	0.11	0.67	RA
E	0.83	1.00	0.83	RA
F	1.00	0.56	1.00	RA

4) Distancias normalizadas desde P(0.43, 0.89, 0.33): A 1.04, **B 0.66**, **C 0.68**, D 0.88, **E 0.65**, F 0.94. Vecinos: E (RA), B (RB), C (RB). Clasificación: **Riesgo Bajo** (2 vs 1).

5) La normalización invierte el resultado: sin normalizar clasificaba Riesgo Alto porque ingresos dominaba; normalizado, las tres variables pesan igual y gana Riesgo Bajo.

6 · Datos desbalanceados

1) Con k=5 es muy probable que los 5 vecinos sean de clase A, ignorando por completo la clase B minoritaria.

2) Pesos por distancia, k más pequeño, técnicas de sampling (oversampling/undersampling) o métricas específicas.

3) Un k más pequeño (1–3) captura mejor la clase minoritaria.

7 · Curse of dimensionality

1) Es la degradación del rendimiento de algoritmos basados en distancia en espacios de alta dimensión: las distancias entre puntos se vuelven poco discriminativas.

2) En alta dimensión los puntos tienden a volverse equidistantes, el volumen crece exponencialmente y la noción de "vecino cercano" pierde sentido.

3) Selección de características, reducción de dimensionalidad (PCA), normalización apropiada y métricas de distancia específicas.

8 · Métricas de distancia

1) Euclidiana: $\sqrt{(4-1)^2 + (6-2)^2 + (9-3)^2} = 9 + 16 + 36 = \sqrt{61} = \mathbf{7.81}$.

2) Manhattan: $|4-1| + |6-2| + |9-3| = 3 + 4 + 6 = \mathbf{13}$.

3) Minkowski (p=3): $\sqrt[3]{(3^3 + 4^3 + 6^3)} = \sqrt[3]{(27 + 64 + 216)} = \sqrt[3]{307} = \mathbf{6.75}$.

4) Minkowski generaliza a las demás: p=1 es Manhattan, p=2 es Euclidiana, p→∞ es Chebyshev.

9 · Pesos por distancia

1) Pesos w = 1/d: clase A → 1/2 = 0.5, clase B → 1/4 = 0.25, clase A → 1/6 = 0.167. Voto de A: $\frac{0.5 + 0.167}{0.5 + 0.25 + 0.167} = 0.667$. Predicción: **A**.

2) Con votación simple también gana A (2 vs 1), pero los pesos reflejan que el vecino B, más lejano, pesa menos.

3) Útil con datos desbalanceados, para dar más importancia a vecinos cercanos o para suavizar la predicción.

10 · Regresión simple

1) Para 75 m², los 3 vecinos: 70 m² → 110000 (d=5), 80 m² → 125000 (d=5), 60 m² → 95000 (d=15). Predicción: $\frac{110000 + 125000 + 95000}{3} = \mathbf{110000\ USD}$.

2) En clasificación se vota la clase mayoritaria; en regresión se promedia el valor objetivo de los vecinos.

11 · Regresión con múltiples variables

1) Normalizando (Publicidad rango 2000, Precio rango 10). El punto (1800, 14) → (0.40, 0.40).

Publicidad norm	Precio norm	Ventas
0.00	0.00	500
0.25	0.20	600
0.50	0.50	700
0.75	0.80	800
1.00	1.00	900

2) Distancias desde (0.40, 0.40): d_1 0.57, d_2 0.25, **d_3 0.14**, d_4 0.53, d_5 0.85.

3) Vecinos: punto 3 (700), punto 2 (600), punto 4 (800).

4) Predicción k=3: $\frac{700 + 600 + 800}{3}$ = **700** ventas.

5) Con k=5: $\frac{500 + 600 + 700 + 800 + 900}{5}$ = **700**. Da igual porque los datos son perfectamente simétricos alrededor del punto.

12 · Outliers en regresión

1) Para 85 m². **Sin outlier:** vecinos 80 (125000), 90 (140000), 70 (110000) → $\frac{125000 + 140000 + 110000}{3}$ = **125000 USD**. **Con outlier:** vecinos 80 (125000), 90 (140000), 95 (500000) → $\frac{125000 + 140000 + 500000}{3}$ = **255000 USD**. El outlier infla la predicción más del 100%.

2) Los outliers pueden ser vecinos cercanos y sesgan el promedio. KNN los sufre más que modelos que ajustan una tendencia global.

3) Usar la mediana en lugar del promedio, detectar y eliminar outliers, pesos por distancia o robust scaling.

13 · El rol de k: sesgo vs. varianza

1) Con k = 1 la frontera es muy irregular y sigue cada punto: el error de **entrenamiento** es 0 (cada punto es su propio vecino), pero el modelo memoriza el ruido. Es **varianza alta** (overfitting): cambia mucho si cambian los datos.

2) Con k = n el modelo ignora la cercanía y predice siempre la clase mayoritaria (o el promedio global en regresión), sin importar la entrada. Es **sesgo alto** (underfitting): la frontera es demasiado simple.

3) Se busca un k intermedio que equilibre ambos, típicamente por validación cruzada (probar varios k y quedarse con el de menor error de validación). k chico → overfitting; k grande → underfitting. El k óptimo está en el punto donde el error de validación se minimiza.

14 · Distancia para variables categóricas

1) Distancia de Hamming de X = (Rojo, Chico) — cuenta atributos distintos: obs 1 (Rojo, Chico) → **0**; obs 2 (Rojo, Grande) → **1** (difiere en Tamaño); obs 3 (Azul, Chico) → **1** (difiere en Color); obs 4 (Verde, Grande) → **2**.

2) Con k = 1: el vecino más cercano es la obs 1 (distancia 0) → **Fruta**. Con k = 3: los 3 más cercanos son obs 1 (Fruta), obs 2 (Fruta) y obs 3 (Juguete) → **Fruta** (2 vs 1).

3) La distancia euclidiana necesita restar valores numéricos; sobre categorías como "Rojo" o "Azul" no hay resta ni orden. Hamming solo pregunta si dos valores son iguales o distintos, que es lo único que tiene sentido con variables categóricas.

15 · Lazy learning y costo computacional

1) Nada de cómputo real: solo **guarda** todo el set de entrenamiento en memoria. No ajusta parámetros ni construye un modelo; por eso se lo llama "lazy".

2) Clasificar una observación cuesta **O(n)**: hay que calcular la distancia a las n observaciones de entrenamiento para encontrar los k vecinos. Todo el trabajo se pospone al momento de predecir.

3) KNN es barato en entrenamiento (solo guardar) y **caro en predicción** (O(n) por consulta). Un árbol es al revés: caro de entrenar, pero predecir es muy barato (recorrer una rama, O(profundidad)).

4) Estructuras de indexado espacial como **KD-trees** o **Ball-trees**, que evitan comparar contra todos los puntos, o métodos aproximados (LSH). También reducir dimensionalidad ayuda.